

Vibe-IC — 全部步驟：Phase → Stage → Step（v2.2.0 • 繁體中文）

一份清單：整條流程的每一步，以 **Phase → Stage → Step** 階層組織，單一連續編號 **1 → 41**（跨 Stage 1-5，自 Stage 1 的 Spec-to-RTL 起算）。Phase 1 的文件生成步驟以字母 **D1-D5** 標示（前置，不計入 1→41）。兩條並行支線 —— Analog（A1-A9）與 Mixed-signal（M1-M4）—— 同時進行。真實來源 = flow/phase1_phase2_phase3.yaml（runner 直接讀它）；更細的程式層標記自動產生於 FLOW_STEPS_GENERATED.md。

本文件受守門測試保護。 programs/tests/test_all_steps_covers_flow.py 會檢查 flow yaml 的每一個步驟 id 在本文件（以及英文正本）都有對應列，且標題的步驟總數與 total_steps 一致。在 yaml 新增或重編步驟時，本文件必須跟著改，否則 CI 失敗 —— 這正是用來防止曾經被漏掉的步驟（Step 18 Design-for-ECO）再次悄悄消失的機制。

Phase → Stage 對照

- **Phase 1** — 規格與文件 → 兩條匯流入口：**Agent 路徑**（PM Agent • IC Expert Agent）• **doc-gen 路徑**（D1-D5）
- **Phase 2** — RTL → 合成 → Stage 1（RTL+驗證）• Stage 2（合成+DFT）
- **Phase 3** — 實體 → Tapeout → Stage 3（實體+簽核）• Stage 4（輸出+Tapeout）• Stage 5（製造與測試）
- **並行** — Analog A1-A9 • Mixed-signal M1-M4

Phase 1 — 規格與文件

Phase 1 有**兩條匯流的入口**，都產出同一批 L1-L23 層文件。刻意**不設 Phase 1a/1b 子編號** —— 依 RFC v2.0 § 8.2，入口差異只是一個輸入子目錄、而非 phase 切分，因為兩條路徑最終都 產出 L1-L13：

- phase1/input_prompt/ —— 自由文字 / 自然語言 → **Agent 路徑**（PM Agent + IC Expert Agent）。
- phase1/input_doc/ —— 既有文件 / 結構化 YAML → **決定性 doc-gen 路徑**（D1-D5）。

兩者匯流到餵給 Phase 2 的 L1-L23 文件。

Agent 路徑 —— phase1/input_prompt/ • 2 個 Agent Skills

當輸入是自由文字時，由兩個 agent 驅動 Phase 1。PM Agent 面對使用者；IC Expert Agent 在其後方運作，**從不直接與使用者對話**。

#	Agent Skill	角色	面對使用者？
1	PM Agent （agents/pm-agent.md）	自然語言前門：NL-ingest → 缺口對話（一次問一題、用使用者的語言）→ 確認 fact graph → 交棒。把使用者的產品語言轉成 L1-L9 事實。	✔ 是
2	IC Expert Agent （agents/ic-expert-agent.md）	PM 後方的矽智財審查者：審查每一層的技術完整性、填入自動決定的預設值（附 auto_decided + reasoning 軌跡）、跨層一致性檢查（L5↔L4 腳位、L6↔L5 訊號、L9↔L5+L6+L8）、套用設計保守原則。 這是 plugin 的 IC 專業知識倉庫 —— 它的逐層審查清單就是領域知識累積之處（每一筆 benchmark-enhancement-capture 的 Bucket-B 學習都落在這裡）。	✗ 否 —— 僅透過 PM Agent

交棒鏈：使用者自由文字 → **PM Agent**（NL-ingest + 缺口對話）→ fact graph → **IC Expert Agent**（審查 / 填值 / 跨層一致性）→ 定稿 L 系列文件。**IC Expert Agent 是 plugin 隨時間累積矽智財知識之處**；PM Agent 則全程讓非專家使用者維持在白話層次。交棒後的事實再走下方同一套 doc-gen 萃取器。

決定性 doc-gen 路徑 —— phase1/input_doc/ • D1-D5（前置 • 不計入 1→41）

#	步驟	工具 / 方式
D1	匯入與文字萃取（prompt 或既有文件 → input_doc/）	phase1_doc_one_shot_runner
D2	產生 L1-L13 核心設計層文件	決定性萃取器
D3	產生 L14-L23 協定 / 時序 / skeleton 文件	overlay 萃取器
D4	協定類別合成 dispatch（81 類）	is_<proto> + <proto>_synth
D5	Coverage / parity 報告	phase1 parity 報告

在 flow yam1 裡，這五個 doc-gen 步驟是單一個受 gate 管控的實體 **D1 — Phase 1 Doc Extraction (17 skills + dialogue entry → L1-L13)**；此處的 D1–D5 是該單一步驟的人類可讀拆解。

Phase 2 — RTL → 合成

Stage 1 — RTL 產生與驗證

#	步驟	工具 / 方式
1	Spec-to-RTL (由 L-docs 撰寫 RTL)	spec-to-rtl skill
2	Lint (RTL + Quartus-unsafe 樣式 + RTL-bug claim schema)	eda_lint + rtl_hygiene_lint
3	CDC / RDC check	cdc-check + rdc-check
4	Simulation (testbench + L10/L12 + Verilator 覆蓋率)	testbench-gen + eda_simulate
5	Formal verification (assertion 證明 + 位元級全棧 tb)	formal-verify + assertion-gen
6	FPGA early prototype + 驗證報告稽核	eda_fpga_compile / eda_fpga_program → .sof

Stage 2 — 合成與 DFT

#	步驟	工具 / 方式
7	Constraint setup (SDC + PVT 矩陣)	constraint-gen → *.sdc
8	SDC validation	SDC lint + sdc_validator_check
9	Synthesis (Yosys → mapped netlist)	eda_synth + synth-doctor
10	Pre-layout STA (multi-corner)	eda_sta (SS/TT/FF)
11	DFT insertion (scan chain + ATPG)	dft-insert + atpg + eda_dft
12	Post-DFT optimization (resynth / buffering)	resynth / buffering
13	Equivalence check (RTL ≡ post-DFT netlist)	equivalence-check + Yosys equiv

Phase 3 — 實體設計 → Tapeout

Stage 3 — 實體設計與簽核

#	步驟	工具 / 方式
14	pre-PnR Yosys gate (synth script template + hilomap 排序稽核)	yosys_script_template_check + yosys_hilomap_required_check
15	Floorplan + PDN	eda_pnr (init)
16	Clock planning	cts-plan skill
17	Placement (global + detailed)	eda_pnr
18	Spare-cell + ECO-prep insertion (Design-for-ECO • 備用單元 + ECO 預備插入)	eda_pnr + spare_cell_coverage_check / spare_cell_preservation_check
19	CTS (Clock Tree Synthesis)	eda_pnr enable_cts=true
20	Post-CTS hold fixing	hold-fix (repair_timing -hold)
21	Routing (global + detailed)	eda_pnr enable_detailed_route=true
22	Parasitic extraction (RC → SPEF)	OpenRCX extract_parasitics
23	Post-route STA (multi-corner multi-mode 簽核)	eda_sta (MMMM)
24	IR drop (static + dynamic)	OpenROAD PSM analyze_power_grid
25	EM check (electromigration 壽命)	PSM -enable_em
26	Antenna check (gate-oxide 保護)	OpenROAD check_antennas + repair_antennas
27	Signal integrity (crosstalk / noise / glitch)	SPEF coupling-cap 篩查
28	Post-layout gate-level sim (Post-Sim + SDF)	eda_simulate
29	Post-layout SPICE verification (關鍵路徑相關性 + analog)	ams-sim + eda_spice

#	步驟	工具 / 方式
30	↻ Physical verification (DRC / LVS / ERC + Density + PERC-equivalent)	KLayout DRC + LVS 簽核鏈 + Magic ERC + perc_equivalent
31	↻ ECO (Engineering Change Order — 修復迴圈)	eco-plan

Stage 4 — 輸出與 Tapeout

#	步驟	工具 / 方式
32	Power analysis (pre/post-layout)	power-analysis
33	Metal fill (ECO-aware density fill 插入)	OpenROAD filler_placement
34	Tapeout checklist (最終簽核確認)	tapeout-checklist + signoff_audit
35	GDSII output	eda_gds + def2gds
36	Foundry handoff (光罩規格 + WAT 計畫 + scribe 佈局 + corner 測試套件)	tapeout-checklist + foundry_handoff_package_check
37	FPGA final sign-off (重編譯 + 板上測試)	recompile + on-board test + attestation

Stage 5 — 製造與測試 (post-fab；僅在收到矽晶時才觸發)

#	步驟	工具 / 方式
38	Fabrication (foundry 光罩組 + 晶圓製造 — 外部)	manufacturing_fab_intake_check
39	Wafer sort / probe test (ATE + probe card)	wafer_sort_yield_check
40	Packaging (封裝：wirebond / FC-CSP / WLCSP)	packaging_intake_check
41	Final test (ATE：functional + parametric + burn-in)	final_test_attestation_check

Stage 5 為條件式：僅在 phase3/stage5_manufacturing/silicon_received.json 存在時才執行。多數 benchmark / 尚未 tapeout 的專案會跳過。

並行支線

Analog A1–A9 (analog_one_shot_runner.py，與 Stage 1–3 並行)

#	步驟	工具 / 方式
A1	Analog spec extraction (規格萃取)	analog-spec-extract → spec.json
A2	Analog topology selection (拓模選擇)	analog-topology-select → topology.md
A3	Analog netlist generation (netlist 產生)	eda_xschem_netlist → <block>.sp
A4	Analog corner sweep (PVT 角點掃描)	eda_spice_corner → corner_results.json
A5	Analog layout (Magic 佈局)	eda_analog_layout → layout.mag / *.gds
A6	Analog physical verification (逐區塊 DRC + LVS，合併前)	analog_a6_block_pv_check
A7	↻ Post-layout resimulation (萃取後再模擬)	analog-extraction-resim → pre_vs_post.json
A8	Hardmacro generation (LEF + Liberty + GDS + Verilog)	analog-hardmacro-gen (餵回 Stage 3)
A9	↻ Co-simulation / HW verification (HIL)	mixed-signal-cosim + eda_spice

Mixed-signal M1–M4 (mixed-signal-cosim skill，存在 analog 區塊時觸發)

#	步驟	工具 / 方式
M1	Mixed-signal 頂層整合 (A+D GDS merge + macro placement)	mixed_signal_merge_check
M2	Mixed-signal 電源域 + level-shifter / isolation 驗證	power_domain_crossing_check + level_shifter_required_check + isolation_cell_required_check
M3	Mixed-signal 驗證 (AMS co-sim + RNM + 介面 SI)	mixed_signal_cosim_check + mixed_signal_interface_si_check
M4	Mixed-signal sign-off (頂層 PV + 最終判定)	mixed_signal_signoff_check

總計

Phase	Stages	Steps
Phase 1 — 規格與文件	兩條入口（Agent • doc-gen）	D1–D5 + PM Agent • IC Expert Agent
Phase 2 — RTL → 合成	Stage 1 • Stage 2	1–13
Phase 3 — 實體 → Tapeout	Stage 3 • Stage 4 • Stage 5	14–41
並行	Analog • Mixed-signal	A1–A9 • M1–M4

41 個循序步驟（Stage 1：1–6 • Stage 2：7–13 • Stage 3：14–31 • Stage 4：32–37 • Stage 5：38–41），外加 **Phase 1**（兩條入口：Agent 路徑 —— PM Agent • IC Expert Agent —— 與 doc-gen 路徑 D1–D5）與兩條並行支線（Analog A1–A9 • Mixed-signal M1–M4）。flow yam! 共計 **54 個實體** = 41 個主軸整數步驟 + A1–A9 + M1–M4 + P0 結構化 RTL 預檢。

預檢：P0（mcp_server_health_check、eda_doctor）。編排器 vibe_ic_one_shot_runner.py 跑 Phase 1 → Phase 2 → Analog → Phase 3。

摘要之外的細節：runner 即時程式層標記見 FLOW_STEPS_GENERATED.md；LVS 簽核鍵與 PERC-equivalent 覆蓋見 PERC_SIGNOFF_MEMO.md。英文正本：ALL_STEPS_v2.2.0.md。